KAVACH

Dispatch Agent

Deep Technical Flowchart & Beginner-Friendly Explanation

A detailed guide to worker eligibility, workload and distance scoring, team selection, vehicle/equipment allocation, ETA estimation, safe reservation, assignment records and notifications.

Purpose	Select eligible workers, check workload and distance, allocate compatible resources, estimate arrival time and coordinate task notifications.
Prototype stack	Python, FastAPI, Pydantic, deterministic scoring and selection, in-memory sample data, simulated notifications.
Key distinction	Required resources are not the same as available resources; finding a worker is not the same as successfully reserving one.
Scope	Dispatch Agent only. Values and policies in examples are illustrative and must be validated by the project team.

This document is a proposed technical design based on the Dispatch Agent responsibilities shown in the KAVACH diagram. The diagram specifies worker availability, workload/distance checks, worker assignment, vehicle/equipment assignment, task notification, assigned workers, vehicle details, ETA and task-completion communication. Specific scoring weights, resource rules and status names below are design suggestions, not source-defined policy.

1. Understand the Dispatch Agent in simple language

Imagine a cleaning plan says a task needs two suitable workers and waste-collection equipment. The Dispatch Agent must find workers who are actually available, check whether they have the right skills, compare workload and travel time, select a feasible team, reserve compatible resources, estimate arrival time and communicate the assignment.

The closest worker is not automatically the best worker. A worker can be nearby but overloaded, unqualified, off shift or already assigned elsewhere. Likewise, an available vehicle may be unsuitable for the job. Dispatch is therefore a constrained resource-allocation problem, not simply a nearest-neighbour search.

2. Recommended Week 1 technology

Component	Simple job	Technical responsibility
Python	Decision and calculation engine.	Eligibility rules, scores, team selection, ETA arithmetic and assignment-state handling.
FastAPI	HTTP front door.	Accept a dispatch request and return structured JSON.
Pydantic	Data checker.	Validate request/response fields, types, enums and constraints.
In-memory repositories	Temporary sample data.	Hold worker, vehicle, equipment, shift, task and assignment records for the prototype.
ETA adapter	Travel-time source.	Use a synthetic travel-time matrix first; a routing provider can be integrated later if needed.
Notification adapter	Communication channel.	Simulate notification events and record status; a real channel can be added later.

Gemini is not necessary for selecting workers, calculating scores, checking resource conflicts or reserving resources. Those steps are more predictable as explicit Python logic. The first prototype can prove dispatch end to end without an LLM.

3. Master technical flowchart

The diagram below presents the full proposed processing sequence. The next page expands the decision and failure paths.



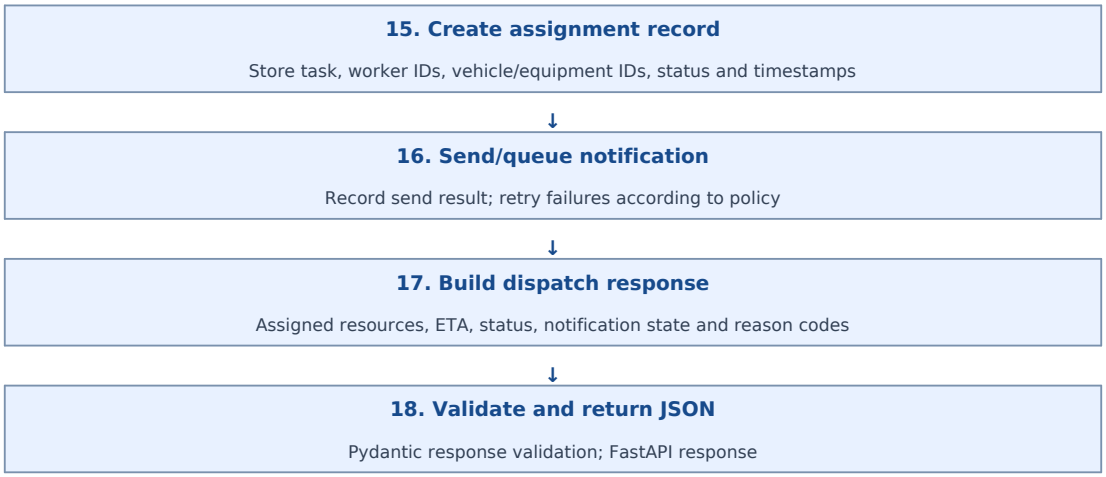


Figure 1. Master flow. Availability must be rechecked before the assignment is committed.

4. Branching flowchart: feasibility and failure handling

A reliable dispatch system must have explicit outcomes when resources are insufficient, data is stale, reservations conflict or notifications fail.



ASSIGNED	PARTIAL / UNASSIGNED	RESOURCE_UNAVAILABLE
Required resources were reserved successfully.	Worker capacity is insufficient to meet the full requirement.	A required compatible vehicle/equipment resource could not be secured.

5. Step-by-step technical explanation

Step 1 — Receive the job

The request describes what the cleaning task needs, not which worker should do it. Include task ID, destination, priority, required skills, team size, equipment and expected duration. Treat missing mandatory fields as an input problem.

Step 2 — Validate the request

Pydantic checks field types and values. Coordinates must be valid if present; worker counts and durations must be non-negative; priority must use an allowed value. Invalid input should not proceed to selection.

Step 3 — Load operational data

Retrieve workers, skills, shifts, workload, locations, vehicles, equipment and current assignments. For the prototype, dictionaries and lists are enough. In a real deployment, these records need reliable persistence and update mechanisms.

Step 4 — Check freshness

A worker location from hours ago may be inaccurate. Availability can change quickly. Use timestamps and a configured freshness policy; refresh stale data or return an uncertainty outcome.

Step 5 — Eligibility filtering

Remove workers who are unavailable, off shift, missing a mandatory skill, over a configured workload limit or known to conflict with the task schedule. Hard constraints must not be bypassed by a high score.

Step 6 — Check worker capacity

Compare the number and skills of eligible workers with the task requirement. If two are required but only one is eligible, return a clear partial/unassigned result unless partial execution is explicitly permitted.

Step 7 — Score candidates

Normalize travel-time, workload, skill-fit and fairness measures onto a common scale. Apply documented weights only after hard eligibility filters. Weights are design parameters that should be tested, not arbitrary facts.

Step 8 — Select a team

For a one-worker job, choose the best feasible candidate. For multi-worker jobs, choose a combination that satisfies team size, required skill coverage and scheduling constraints. A simple greedy algorithm can be a Week 1 baseline.

Step 9 — Check scheduling conflicts

Confirm the workers can cover the task duration, are not double-booked and can finish within their shifts. Check team availability over the time window, not just at one instant.

Step 10 — Select vehicle/equipment

Filter resources by type, capacity, readiness, location, compatibility and time-window availability. Do not assign an unavailable or unsuitable vehicle merely because it is nearby.

Step 11 — Estimate ETA

Use a synthetic worker-to-site travel-time matrix in the prototype. Later, a road-routing service may provide better estimates. Distinguish estimated travel duration from a clock-time arrival timestamp.

Step 12 — Recheck and reserve

The chosen worker or vehicle may have been taken by another request. Recheck availability at commit time and reserve resources atomically. If reservation fails, recalculate or return a controlled failure.

Step 13 — Create the assignment record

Store a unique assignment ID, task ID, worker IDs, vehicle/equipment IDs, timestamps and assignment status. The record becomes the source of truth for what the system actually assigned.

Step 14 — Notify workers

Send or queue a notification through an adapter. In Week 1, simulate the send and record it. In a real integration, track pending, sent, failed and—where supported—acknowledged states.

Step 15 — Return dispatch result

Return assigned resources, ETA, assignment status, notification status and reason codes. Do not claim a resource is assigned until reservation succeeds or a notification was delivered when delivery has not been confirmed.

6. Example request and data models

Illustrative dispatch request JSON

```
{
  "task_id": "TASK-104",
  "incident_id": "INC-104",
  "required_workers": 2,
  "required_skills": ["WASTE_COLLECTION"],
  "location": {"latitude": 22.7196, "longitude": 75.8577},
  "required_resources": ["WASTE_COLLECTION_EQUIPMENT"],
  "estimated_task_duration_minutes": 45,
  "priority": "MEDIUM"
}
```

Illustrative Pydantic model outline

```
from pydantic import BaseModel, Field
from typing import Literal

class Coordinates(BaseModel):
    latitude: float = Field(ge=-90, le=90)
    longitude: float = Field(ge=-180, le=180)

class DispatchRequest(BaseModel):
    task_id: str
    incident_id: str
    required_workers: int = Field(ge=1)
    required_skills: list[str]
    location: Coordinates
    required_resources: list[str]
    estimated_task_duration_minutes: int = Field(ge=0)
    priority: Literal["LOW", "MEDIUM", "HIGH"]
```

The final schema should be shared and versioned before team members implement against it. This outline is not a complete production model.

7. Worker scoring example

One possible illustrative score is S = 0.40T + 0.30W + 0.20K + 0.10F, where T is travel-time suitability, W workload suitability, K skill fit and F fairness. Normalize each component to 0–100. The weights are examples only; tune them with stakeholder input and test scenarios. Required skills and availability remain hard filters.

```
def is_eligible(worker, required_skills):
    if not worker["available"]:
        return False
    if not set(required_skills).issubset(set(worker["skills"])):
        return False
    if worker["active_tasks"] >= worker["max_active_tasks"]:
        return False
    return True
```

This function is a minimal illustration. A full implementation must also consider shifts, task overlap, data freshness, location and required team skills.